
Real-Time Cryptocurrency Market Dashboard

1. Project Overview

1.1 Description

This project is a real-time web-based dashboard built with Python (Streamlit) that fetches live cryptocurrency market data from the CoinGecko REST API and visualizes it interactively. The dashboard displays global market statistics (total market cap, 24h trading volume, Bitcoin dominance), top-performing coins with price change indicators, an interactive 7-day price history chart for any selected coin, a market cap comparison of the top 10 cryptocurrencies, and a detailed sortable data table showing hourly, daily, and weekly price movements.

Data is automatically refreshed at user-defined intervals using an auto-refresh mechanism, and API responses are cached to optimize performance and stay within the free-tier rate limit (30 calls per minute). The application demonstrates key data-engineering concepts: REST API consumption, JSON parsing, data caching, and real-time data visualization using Plotly and Pandas.

1.2 Technology Stack

The dashboard is built on a modern Python data stack selected for developer productivity, real-time interactivity, and broad community support. Each component plays a specific role in the data pipeline:

- Python 3 - Streamlit - Plotly - Pandas - Requests

Streamlit provides the reactive web framework, Plotly powers the interactive charts, Pandas handles tabular transformations, and the Requests library manages authenticated calls to the CoinGecko REST endpoint. Together they enable a fully functional dashboard in a single Python file with no front-end code required.

1.3 Data Source

CoinGecko API v3 (free Demo plan) - <https://www.coingecko.com/en/api>

The CoinGecko Demo plan provides 30 requests per minute at no cost, which is sufficient for an auto-refreshing dashboard when combined with a 30-second cache. Authentication uses the x-cg-demo-api-key header. The endpoints used are /global for aggregate market statistics and /coins/markets for per-coin price, market cap, volume, and 7-day sparkline data.

2. Output & Deliverables

2.1 How to Run

Execute the following commands from a terminal in the project directory. A Python 3.8+ environment is recommended.

```
pip install streamlit streamlit-autorefresh requests pandas plotly
streamlit run crypto_dashboard.py
Dashboard opens at http://localhost:8501
```

2.2 Dashboard Sections

The dashboard is organized into seven logical sections, each serving a distinct analytical purpose. The table below summarizes what the end user sees on screen.

#	Section	Output Shown
1	Sidebar (Controls)	Password-masked API key field, auto-refresh slider (30-300 sec), link to get a free API key
2	Header	Dashboard title with live clock showing last-updated time and refresh interval
3	Global KPI Cards (4)	Total Market Cap (e.g. \$1.72T +2.35% 24h), 24h Volume, BTC Dominance, Active Coins
4	Top Coins (4 cards)	BTC, ETH, USDT, BNB - current price and green/red 24h change
5	7-Day Price Chart	Interactive line chart for the selected coin; green fill if up over the week, red if down; caption with 7d change, high, low

#	Section	Output Shown
6	Market Cap Bar Chart	Top 10 coins on log scale, bars colored green/red by 24h performance, formatted labels (\$1.3T, \$450B...)
7	Full Market Data Table	Sortable table of 15 coins: Price, 1h %, 24h %, 7d %, Market Cap, 24h Volume

2.3 Key Behaviors

The dashboard implements several real-time and resilient behaviors that distinguish it from a static report:

- The page auto-refreshes every N seconds - prices, charts and the clock update without manual reload.
- Changing the coin in the dropdown instantly redraws the 7-day chart.
- Missing/invalid API key shows a warning; rate limit shows guidance - the app never crashes.

3. Full Source Code

The complete implementation resides in a single file, `crypto_dashboard.py`. It is reproduced below in full for reference and audit. The code is organized into clearly commented blocks: sidebar controls, API helpers, data loading with error handling, global KPIs, top coin cards, the 7-day price chart, the market cap bar chart, and the full sortable data table.

```
import streamlit as st
import requests
import pandas as pd
import plotly.graph_objects as go
from datetime import datetime
from streamlit_autorefresh import st_autorefresh

# FREE KEY: https://www.coingecko.com/en/api
DEFAULT_KEY = "PASTE_YOUR_KEY_HERE"

BASE = "https://api.coingecko.com/api/v3"
NUM_COINS = 15

GREEN = "#26a69a"
RED = "#ef5350"

st.set_page_config(
    page_title="Live Crypto Dashboard",
    layout="wide",
)

# ----- SIDEBAR -----
```

```

with st.sidebar:
    st.header("Controls")
    api_key = st.text_input(
        "CoinGecko API key",
        value=DEFAULT_KEY,
        type="password",
    )
    refresh = st.slider(
        "Auto-refresh (seconds)",
        30, 300, 60,
        step=30,
    )
    st.markdown("[Get a free key](https://www.coingecko.com/en/api)")
    st.caption("Free plan: 30 calls/min")

st_autorefresh(interval=refresh * 1000, key="refresh")

st.title("Live Crypto Market Dashboard")
st.caption(
    f"Last updated: {datetime.now().strftime('%H:%M:%S')}"
    f" | refresh every {refresh}s"
)

HEADERS = {}
if api_key and "PASTE" not in api_key:
    HEADERS = {"x-cg-demo-api-key": api_key}

# ----- API HELPERS -----
def _get(url, params=None):
    r = requests.get(url, params=params, headers=HEADERS, timeout=15)
    if r.status_code != 200:
        try:
            reason = r.json()["status"]["error_message"]
        except Exception:
            reason = r.text[:120]
        msg = f"HTTP {r.status_code} - {reason}"
        raise RuntimeError(msg)
    return r.json()

@st.cache_data(ttl=30, show_spinner=False)
def fetch_global(_key):
    return _get(f"{BASE}/global")["data"]

@st.cache_data(ttl=30, show_spinner=False)
def fetch_markets(_key):
    data = _get(f"{BASE}/coins/markets", params={
        "vs_currency": "usd",
        "order": "market_cap_desc",
        "per_page": NUM_COINS,
        "page": 1,
        "sparkline": "true",
        "price_change_percentage": "1h,24h,7d",
    })
    return pd.DataFrame(data)

def fmt_usd(n):
    if n >= 1e12:
        return f"${n/1e12:.2f}T"
    if n >= 1e9:
        return f"${n/1e9:.2f}B"
    if n >= 1e6:
        return f"${n/1e6:.2f}M"

```

```

        return f"${n:,.0f}"

def safe(v):
    return 0 if pd.isna(v) else v

# ----- LOAD DATA -----
try:
    g = fetch_global(api_key)
    df = fetch_markets(api_key)
except Exception as e:
    st.error(f"API error: {e}")
    if "401" in str(e):
        st.info("Invalid key. Paste your CoinGecko key in sidebar.")
    if "429" in str(e):
        st.info("Rate limited. Increase the refresh interval.")
    st.stop()

if "PASTE" in api_key:
    st.warning(
        "No API key set. Get one free: "
        "https://www.coingecko.com/en/api"
    )

# ----- GLOBAL KPIs -----
mc = g["total_market_cap"]["usd"]
mcc = g["market_cap_change_percentage_24h_usd"]
vol = g["total_volume"]["usd"]
btc = g["market_cap_percentage"]["btc"]
coins = g["active_cryptocurrencies"]

c1, c2, c3, c4 = st.columns(4)
c1.metric("Total Market Cap", fmt_usd(mc), f"{mcc:+.2f}% (24h)")
c2.metric("24h Volume", fmt_usd(vol))
c3.metric("BTC Dominance", f"{btc:.1f}%")
c4.metric("Active Coins", f"{coins:,}")

# ----- TOP COIN CARDS -----
st.subheader("Top Coins")
cols = st.columns(4)
for col, (_, row) in zip(cols, df.head(4).iterrows()):
    chg = safe(row["price_change_percentage_24h_in_currency"])
    col.metric(
        label=row["symbol"].upper(),
        value=f"${row['current_price']:, .2f}",
        delta=f"{chg:+.2f}% (24h)",
    )

# ----- 7-DAY CHART -----
st.subheader("7-Day Price Chart")
coin = st.selectbox("Coin", df["name"])
row = df[df["name"] == coin].iloc[0]
prices = row["sparkline_in_7d"]["price"]
chg7 = safe(row["price_change_percentage_7d_in_currency"])
times = pd.date_range(
    end=datetime.now(),
    periods=len(prices),
    freq="60min",
)

if chg7 >= 0:
    line_color = GREEN
    fill_rgb = "38,166,154"
else:
    line_color = RED
    fill_rgb = "239,83,80"

```

```

fill_rgba = f"rgba({fill_rgb},0.12)"

fig = go.Figure()
fig.add_trace(go.Scatter(
    x=times,
    y=prices,
    mode="lines",
    line=dict(color=line_color, width=3),
    fill="tozero",
    fillcolor=fill_rgba,
))
fig.update_layout(height=320, margin=dict(l=10, r=10, t=20, b=10))
fig.update_yaxes(title="USD")
fig.update_xaxes(tickformat="%b %d")
st.plotly_chart(fig, use_container_width=True)

st.caption(
    f"7d change: {chg7:+.2f}% | "
    f"High ${max(prices):,.2f} | Low ${min(prices):,.2f}"
)

# ----- MARKET CAP BAR -----
st.subheader("Market Cap - Top 10")
top = df.head(10)
chg24 = top["price_change_percentage_24h_in_currency"].apply(safe)
colors = []
for v in chg24:
    colors.append(GREEN if v >= 0 else RED)
labels = [fmt_usd(v) for v in top["market_cap"]]

fig2 = go.Figure()
fig2.add_trace(go.Bar(
    x=top["name"],
    y=top["market_cap"],
    marker_color=colors,
    text=labels,
    textposition="outside",
))
fig2.update_layout(height=380, margin=dict(l=10, r=10, t=20, b=10))
fig2.update_yaxes(type="log", title="Market Cap (log)")
st.plotly_chart(fig2, use_container_width=True)

# ----- FULL TABLE -----
st.subheader("Full Market Data")
show = df[[
    "name",
    "symbol",
    "current_price",
    "price_change_percentage_1h_in_currency",
    "price_change_percentage_24h_in_currency",
    "price_change_percentage_7d_in_currency",
    "market_cap",
    "total_volume",
]].copy()
show.columns = [
    "Coin", "Symbol", "Price",
    "1h %", "24h %", "7d %",
    "Market Cap", "Volume 24h",
]
for c in ["1h %", "24h %", "7d %"]:
    show[c] = show[c].apply(safe)

st.dataframe(
    show,
    column_config={
        "Price": st.column_config.NumberColumn(format="$%.2f"),
        "1h %": st.column_config.NumberColumn(format="%+.2f%"),
        "24h %": st.column_config.NumberColumn(format="%+.2f%"),
    },
)

```

```

    "7d %": st.column_config.NumberColumn(format="%.2f%%"),
    "Market Cap": st.column_config.NumberColumn(format="$.0f"),
    "Volume 24h": st.column_config.NumberColumn(format="$.0f"),
  },
  hide_index=True,
  use_container_width=True,
)

```

4. Output

